

AI Agent Orchestration Frameworks

A practical comparison of LangGraph, OpenAI Agents SDK, Anthropic SDK, and CrewAI

1. What Agent Orchestration Means

An AI agent is a software system that uses a language model to reason about a task, select actions, call tools, observe results, and continue until it reaches an outcome. A single model call can answer a question, but an agentic application often needs multiple steps, external tools, state, retries, routing, memory, human approval, and coordination between specialized workers. Agent orchestration is the layer that controls this workflow.

Orchestration frameworks solve similar problems in different ways. Some emphasize explicit graphs and state machines. Others provide lightweight agent loops and tool calling. Some are designed around teams of role-based agents. The best choice depends on how much control, observability, persistence, and abstraction a project requires.

LangGraph: Explicit Stateful Workflows

LangGraph is designed for building stateful, long-running agent workflows as graphs. A developer defines nodes that perform work and edges that determine what happens next. State moves through the graph and can contain messages, structured outputs, counters, retrieved documents, human feedback, or other application data. Conditional edges can route execution based on that state.

This model is useful when the workflow must be visible and controllable. For example, a document pipeline can parse a PDF, review chunks in parallel, create a learning plan, pause for human approval, generate questions, retrieve evidence, and build a quiz. Each stage can be represented as a node rather than hidden inside one autonomous loop.

LangGraph also supports checkpointing and human-in-the-loop patterns. Checkpoints allow workflow state to persist across execution boundaries, while interrupts allow execution to pause so a person can inspect or modify state before the graph continues. This makes LangGraph especially well suited to workflows where reliability and control matter more than minimizing code.

OpenAI Agents SDK: Agent Loops with Built-In Primitives

The OpenAI Agents SDK provides a comparatively direct way to define agents, give them instructions and tools, and run agent loops. An agent can call functions, use tools, and hand work to another agent. The SDK is useful when the desired architecture maps naturally to agents, tools, handoffs, and execution traces without requiring an explicit graph for every transition.

A key difference from LangGraph is the level of orchestration abstraction. With LangGraph, the developer commonly models the workflow itself as a graph. With the OpenAI Agents SDK, the developer can often describe agents and capabilities and let the runner manage the underlying loop. This can reduce orchestration code for straightforward agent applications.

2. Anthropic SDK and CrewAI

Anthropic SDK: Model and Tool Integration

Anthropic's SDK gives developers programmatic access to Claude models and the primitives needed to build applications around them. Developers can send messages, provide tools, process tool-use requests, return tool results, and continue the interaction. This is enough to implement an agent loop, but the SDK itself is closer to a model API toolkit than a complete graph-style orchestration framework.

That lower-level approach can be an advantage. A team can design its own control loop, persistence strategy, routing rules, approval logic, and application architecture rather than adopting a larger orchestration abstraction. The tradeoff is that more workflow behavior may need to be implemented by the application.

This distinction is important when comparing SDKs and frameworks. An SDK can provide excellent model access and tool-use primitives without prescribing how a complex multi-step workflow should be represented. A framework such as LangGraph provides additional structure for coordinating those steps.

CrewAI: Role-Based Multi-Agent Collaboration

CrewAI emphasizes teams of agents with roles, goals, tasks, and collaboration patterns. Instead of starting from nodes and edges, a developer can think in terms of a crew: for example, a researcher, an analyst, a writer, and a reviewer. Tasks can be assigned to those agents and coordinated as a larger process.

This abstraction can make multi-agent prototypes intuitive because the software architecture mirrors the way people describe a team. A research workflow might assign information gathering to one agent, analysis to another, and final writing to a third. CrewAI can therefore be attractive when role specialization is central to the application.

The tradeoff is abstraction. Role-based agent systems can be easy to describe, but developers still need to reason carefully about deterministic routing, shared state, failure handling, repeated work, cost, and whether multiple agents are actually necessary. A multi-agent architecture should solve a real decomposition problem rather than simply adding more model calls.

Control Versus Abstraction

These tools can be placed on a rough spectrum. LangGraph emphasizes explicit workflow control. OpenAI Agents SDK emphasizes agent execution primitives and handoffs. Anthropic SDK provides flexible model and tool-use building blocks from which a custom loop can be built. CrewAI emphasizes a higher level team-and-role metaphor. None of these approaches is universally better; each exposes different parts of the orchestration problem to the developer.

A useful design question is: where should control live? If every transition must be inspectable, a graph is attractive. If a small number of agents can autonomously decide when to use tools or hand off work, an agent SDK may be simpler. If the application needs a custom Claude-centered execution loop, Anthropic's SDK provides the primitives. If the problem naturally resembles a team of specialists, CrewAI's role model may fit well.

3. Comparing the Frameworks in Practice

Approach	Primary mental model	Strong fit	Main tradeoff
LangGraph	Nodes, edges, state, routing	Controlled, stateful, multi-step workflows	More orchestration structure to design
OpenAI Agents SDK	Agents, tools, handoffs, runner	Direct agent applications and delegation	Less graph-like control by default
Anthropic SDK	Messages, Claude, tools, custom loop	Custom Claude-centered applications	More orchestration may be application-owned
CrewAI	Roles, agents, tasks, crews	Role-oriented multi-agent workflows	Higher abstraction can hide workflow complexity

Example: Building a Research Agent

Imagine an application that receives a company name, researches public sources, analyzes the evidence, and produces a final briefing. In LangGraph, research, analysis, review, and report generation could be separate nodes connected by explicit edges. Parallel searches could fan out and later merge into shared state. A human approval node could pause the workflow before publication.

With the OpenAI Agents SDK, a research agent could receive search tools and then hand the collected material to an analyst or writer agent. The orchestration can focus on agent responsibilities and handoffs instead of representing every transition as a graph.

With the Anthropic SDK, the application could implement its own loop around Claude: send the research request, execute requested tools, return results, track application state, and continue until a final answer is produced. This offers flexibility but places more responsibility on the surrounding application.

With CrewAI, the same system could be modeled as a crew containing a researcher, analyst, and report writer, each with a defined role and task. This makes specialization easy to communicate, although the team must still decide how information is shared and how failures or low-quality results are handled.

Choosing an Approach

Choose based on the workflow rather than the popularity of the framework. LangGraph is compelling when you need explicit state, branching, persistence, parallel execution, and human checkpoints. OpenAI Agents SDK is attractive when agents, tools, and handoffs describe the application cleanly. Anthropic SDK is a good foundation when you want direct Claude integration and are comfortable owning more of the control loop. CrewAI is useful when the task genuinely benefits from a team of specialized agents.

In production, the framework is only one part of the system. Reliable agent applications also need evaluation, observability, error handling, security controls, cost management, evidence grounding, and clear rules for when a human must intervene. Good orchestration makes those behaviors understandable instead of hiding them behind a single autonomous agent.